

> Identificar la Arquitectura

El rendimiento SIMD, va a depender de que instrucciones y tipos de datos están disponibles. Por tanto lo primero es verificar el modelo de CPU que estamos usando.

[] ↳ 12 cells hidden

✓ SIMD: Instrucciones SSE y AVX

Se puede encontrar una descripción completa de las instrucciones SSE y AVX en el enlace:

<https://software.intel.com/sites/landingpage/IntrinsicsGuide/>

Como referencia indicar que cualquier CPU producida después de 2000 admite SSE hasta 4.2.

AVX y especialmente AVX2 son tecnologías más recientes; consulte Wikipedia para obtener una lista de procesadores compatibles: https://en.wikipedia.org/wiki/Advanced_Vector_Extensions

Antes de empezar con las funciones intrínsecas relacionadas con las matemáticas proporcionadas por AVX y AVX2 es importante entender los siguiente puntos:

- Tipos de datos
- Convenciones de nomenclatura de funciones

Tipos de datos SIMD

Algunos *intrinsics* aceptan tipos de datos tradicionales como ints o floats, pero la mayoría opera con tipos de datos que son específicos de AVX y AVX2. Hay seis tipos de vectores principales enumerados en la Tabla 1

Tabla 1: Tipos de datos AVX/AVX2

Type	Explanation
__m128	vector de 128-bit que contiene 4 floats
__m128d	vector de 128-bit que contiene 2 doubles
__m128i	vector de 128-bit que contiene integers
__m256	vector de 256-bit que contiene 8 floats
__m256d	vector de 256-bit que contiene 4 doubles
__m256i	vector de 256-bit que contiene integers

Cada tipo comienza con dos guiones bajos, una m y el ancho del vector en bits. AVX512 admite

tipos de vectores de 512 bits que comienzan con `_m512`, pero los vectores AVX/AVX2 no superan los 256 bits.

Si un tipo de vector termina en `d`, contiene *doubles* y si no tiene sufijo, contiene *floats*. Puede parecer que los vectores `_m128i` y `_m256i` deben contener enteros, pero no es así. Un tipo de vector entero puede contener cualquier tipo de número entero, desde *chars* hasta *shorts* y *longs* sin signo. Es decir, un `_m256i` puede contener 32 *chars*, 16 *shorts*, 8 *ints*, or 4 *longs*. Estos números enteros pueden tener o no signo.

Convención de nombres de funciones

Los nombres de los *intrinsics* AVX/AVX2 pueden resultar confusos al principio, pero la convención de nomenclatura es realmente sencilla. Una función intrínseca genérica de AVX/AVX2 se proporciona de la siguiente manera:

`_mm<bit_width>_<nombre>_<tipo_datos>`

Las partes de este formato se interpretan de la siguiente manera:

- `<bit_width>` identifica el tamaño del vector devuelto por la función. Para vectores de 128 bits, está vacío. Para vectores de 256 bits, se establece en 256.
- `<nombre>` describe la operación realizada por el intrínseco
- `<tipo_datos>` identifica el tipo de datos de los argumentos principales de la función. Esta última parte, `<tipo_datos>`, es un poco complicada. Identifica el contenido de los valores de entrada y se puede establecer en cualquiera de los siguientes valores:
 - `ps` - vector contiene floats (`ps` indica packed single-precision)
 - `pd` - vector contiene doubles (`pd` indica packed double-precision)
 - `epi8/epi16/epi32/epi64` - vector contiene 8-bit/16-bit/32-bit/64-bit enteros con signo
 - `eput8/eput16/eput32/eput64` - vector contiene 8-bit/16-bit/32-bit/64-bit * enteros sin signo
 - `si128/si256` - vector de 128-bit sin especificar o vector de 256-bit
 - `m128/m128i/m128d/m256/m256i/m256d` - identifica los tipos de vectores de entrada cuando son diferentes al tipo del vector devuelto

✓ Ejercicios

Ejercicio 1.1

Considera la función `_mm256_srlv_epi64`. Incluso si no sabe lo que significa `srlv`, ¿Qué se puede decir de la salida y de los argumentos de entrada de la función?

Es una función que devuelve un vector de 256 bits con el nombre de la operación que va a realizar que es `srlv` (desplazamiento lógico hacia la derecha) y el tipo de datos de los argumentos de entrada es vector que contiene 64 bit enteros con signo.

>

✓ Ejercicio 1.2

Considera la función `_mm_testnzc_ps`. Incluso si no sabe lo que significa `testnzc`, ¿Qué se puede decir de la salida y de los argumentos de entrada de la función?

Es una función que devuelve un vector de 128 bits, realiza la operación `tetnzc` y los argumentos de entrada son de tipo vector que contiene floats.

>

Ejemplo que utiliza AVX Intrinsic

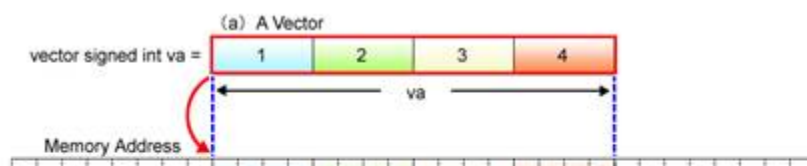
Para crear una aplicación que utilice elementos intrínsecos de AVX, no es necesario vincular ninguna biblioteca. Pero se debe incluir el archivo de encabezado `immintrin.h`. Este encabezado incluye otros encabezados que asignan funciones AVX/AVX2 a instrucciones.

[] ↳ 7 cells hidden

Alineación de datos en memoria: SIMD Acceso a datos

Las operaciones SIMD procesan datos según el principio de "múltiples datos = múltiples datos + múltiples datos". Esto significa que las operaciones SIMD pueden procesar una mayor cantidad de datos por instrucción que las operaciones escalares, lo que permite reducir el número de instrucciones necesarias a ejecutar y, por tanto, el tiempo necesario para su procesamiento.

Para que el rendimiento no se vea penalizado por el acceso a memoria, es conveniente que el acceso a los datos sea lo más eficiente posible, y esto se consigue con datos alineados en memoria.



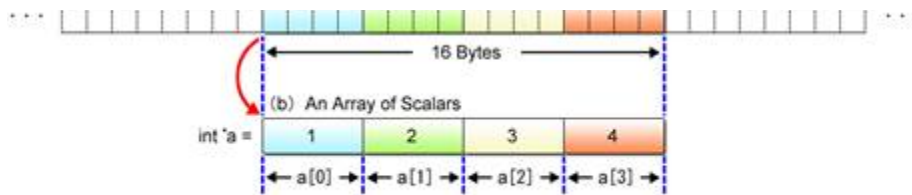
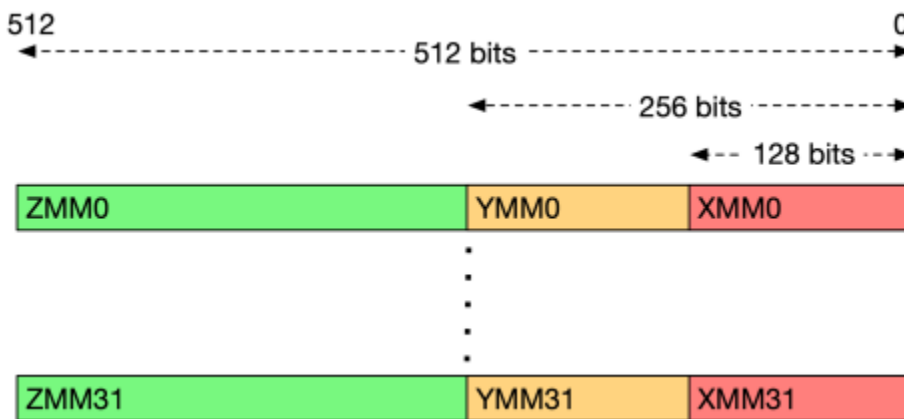


Fig. 2.5: Relationship between Scalar Array and Vector

[] ↳ 5 cells hidden

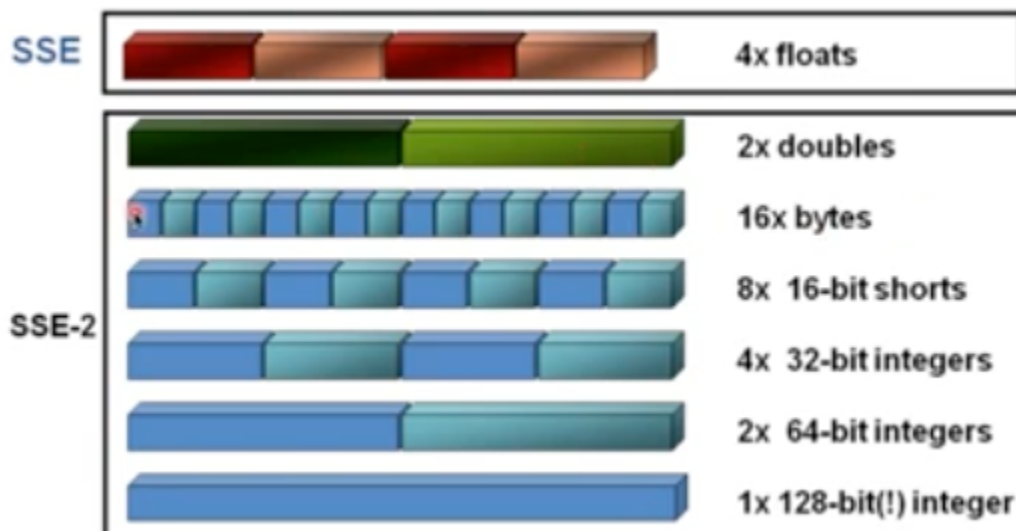
✓ Registros SIMD

Registros vectoriales en aquitectura x86-64 con soporte AVX512



En las primeras generaciones SSE y SSE2 los registros vectoriales son de 128 bit.

Figura SIMD registros



▼ Ejercicios

- E1.3 En un registro SIMD de 128 bit, ¿Cuántos enteros se pueden almacenar?
 - E1.4 En un registro SIMD de 128 bit, ¿Cuántos números reales de precisión simple (float) se pueden almacenar?
 - E1.5 En un registro SIMD de 128 bit, ¿Cuántos números reales de doble precisión (double) se pueden almacenar?
-
- E1.3 En un registro SIMD de 128 bit se pueden almacenar 4 enteros.
 - E1.4 En un registro SIMD de 128 bits se pueden almacenar 4 floats.
 - E1.5 En un registro SIMD de 128 bits se pueden almacenar 2 double.

▼ Ejercicios registros SIMD

En un procesador con extensiones SIMD se utilizan registros de 256 bits.

- E1.6 ¿Cuántos números reales de doble precisión pueden almacenar?
 - E1.7 ¿Cuántos caracteres se pueden almacenar?
 - E1.8 ¿Cuántos *píxeles* se pueden almacenar? (un pixel son 3 bytes para almacenar rojo-verde y azul)
-
- E1.6: Se pueden almacenar 32 numeros reales de doble precisión.
 - E1.7: Se pueden almacenar 256 carácteres.
 - E1.8: Se pueden almacenar 85.33 píxeles.

▼ Ejercicios registros YMM

Los registros YMM son de 256 bits

- E1.9 ¿Cuantos enteros se pueden almacenar en un registro YMM?
 - E1.10 Suponga un vector de 32 doubles, (double vect[32]), ¿ Cuantos registros se necesitan para procesarlo utilizanco instrucciones SIMD?
-
- E1.9: En registros YMM se pueden almacenar 8 enteros.
 - E1.10: Se necesitan 8 registros para procesarlo utilizando instrucciones SIMD.

Evaluar tamaño del problema a resolver en función de la

✓ Evaluar tamaño del problema a resolver en función de la capacidad de memoria y su jerarquía

Elegir distinto tamaños de vector cumpliendo_

- Supere el tamaño de de L1d
- Supere el tamaño de L2
- Supere el tamaño de L3 y con tamaños multiplos del número de elementos SIMD a procesar y cuando no lo sean-

[] ↳ 2 cells hidden

Evaluar el resultado de las opciones de compilación

El compilador transforma el código fuente en un binario legible por máquina. Este paso es necesario para ejecutar el código en una CPU. Además de traducir el código fuente a binario, el compilador también puede optimizar el código para reducir el tiempo de ejecución y el consumo de energía. Para lograr esto, el compilador proporciona opciones de optimización. Los más conocidos empiezan por -O y definen un nivel de optimización global:

- O0 Sin optimización,
- O1 Habilita una serie de optimizaciones para reducir el tamaño binario y el tiempo de ejecución, mientras mantiene el tiempo de compilación relativamente bajo.
- O2 Permite todas las optimizaciones posibles (excepto aquellas que requieren un compromiso entre eficiencia y tamaño binario), esta opción requiere un tiempo de compilación más largo que -O1,
- O3 Permite todas las optimizaciones posibles que no comprometan la precisión del cálculo ni el tiempo de compilación. aumenta en comparación con -O2.

Estos diferentes niveles de optimización son válidos para compiladores GNU (gcc) y compiladores Clang. Pueden variar según el compilador que utilices, por lo que es responsabilidad del desarrollador familiarizarse con su compilador. Para obtener más detalles sobre las distintas optimizaciones del compilador, visita <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>

✓ Ejemplo: Producto escalar de dos vectores con elementos reales (double)

La compilación del código SIMD por defecto se realiza con `CFLAGS := -O3 -march=native`

Consulta el funcionamiento de las funciones utilizadas en el código que se proporciona de ejemplo en el [enlace https://software.intel.com/sites/landingpage/IntrinsicsGuide/](https://software.intel.com/sites/landingpage/IntrinsicsGuide/)

Concretamente, las funciones utilizadas son: En el producto elemento a elemento de los vectores:

- `__m256d x = _mm256_load_pd(a+4*ii);`
- `__m256d y = _mm256_load_pd(b+4*ii);`
- `__m256d z = _mm256_mul_pd(x,y);`
- `sum_vec = _mm256_add_pd(sum_vec, z);`

En la reducción del vector:

- **`m256d temp`** = `(m256d) _mm256_hadd_pd(input, input);`
- `__m256d temp = _mm256_hadd_pd(input, input);`
- `__m128d sum_high = _mm256_extractf128_pd(temp, 1);`
- `__m128d result = _mm_add_pd(sum_high, _mm256_castpd256_pd128(temp));`

```
%%writefile exdot_product.c
/* Compute the dot product of two (properly aligned) vectors. */
/* Limitado a tamaños reserva mem de 512215x8xbytesx2 (aprox 8 MBytes) */

#define MAX_ITER 5
#define MAX_N 512215
#define NUM_TAM 12

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <sys/time.h>
#include <immintrin.h>

double slow_dot_product(const double *a, const double *b, int num_elem) {
    double answer = 0.0;
    for(int ii = 0; ii < num_elem; ++ii)
        answer += a[ii]*b[ii];
    return answer;
}

double reverse_slow_dot_product(const double *a, const double *b, int num_elem) {
    double rev_answer = 0.0;
    for(int ii = 0; ii < num_elem; ++ii)
        rev_answer += a[num_elem-ii-1]*b[num_elem-ii-1];
}
```

```
    return rev_answer;
}

/* Horizontal add works within 128-bit lanes. Use scalar ops to add
 * across the boundary. */
double reduce_vector1(__m256d input) {
    __m256d temp = (__m256d) _mm256_hadd_pd(input, input);
    return ((double*)&temp)[0] + ((double*)&temp)[2];
}

/* Another way to get around the 128-bit boundary: grab the first 128
 * bits, grab the lower 128 bits and then add them together with a 128
 * bit add instruction. */
double reduce_vector2(__m256d input) {
    __m256d temp = _mm256_hadd_pd(input, input);
    __m128d sum_high = _mm256_extractf128_pd(temp, 1);
    __m128d result = _mm_add_pd(sum_high, _mm256_castpd256_pd128(temp));
    return ((double*)&result)[0];
}

double dot_product(const double *a, const double *b, int num_elem) {
    __m256d sum_vec = _mm256_set_pd(0.0, 0.0, 0.0, 0.0);

    /* Add up partial dot-products in blocks of 256 bits */
    for(int ii = 0; ii < num_elem/4; ++ii) {
        __m256d x = _mm256_load_pd(a+4*ii);
        __m256d y = _mm256_load_pd(b+4*ii);
        __m256d z = _mm256_mul_pd(x,y);
        sum_vec = _mm256_add_pd(sum_vec, z);
    }

    /* Find the partial dot-product for the remaining elements after
     * dealing with all 256-bit blocks. */
    double final = 0.0;
    for(int ii = num_elem-num_elem%4; ii < num_elem; ++ii)
        final += a[ii] * b[ii];

    return reduce_vector2(sum_vec) + final;
}

int main(int argc, char **argv) {

    __attribute__((aligned (32))) double a[MAX_N], b[MAX_N];
    struct timeval fin,ini;
    long int tiempo[MAX_ITER],tiempodot[MAX_ITER];
    double answer[MAX_ITER], answerdot[MAX_ITER], rev_answerdot[MAX_ITER];;
    float acel;

    int tam[NUM_TAM];
```



```

int tam0 = 8191; tam[0]=tam0;
int tam1 = 8192; tam[1]=tam1;
int tam2 = 16383; tam[2]=tam2;
int tam3 = 16384; tam[3]=tam3;
int tam_L1d = 32768; tam[4]=tam_L1d;
int tam_L1di = 32765; tam[5]=tam_L1di;
int tam6= 131072; tam[6]=tam6;
int tam7= 131070; tam[7]=tam7;
int tam_L2 = 262144; tam[8]=tam_L2;
int tam_L2i = 262141; tam[9]=tam_L2i;
int tam10 = 512215; tam[10]=tam10;
int tam11 = 512212; tam[11]=tam11;


int N = MAX_N;
// asignación de valores
for(int ii = 0; ii < N; ++ii){
    a[ii] = b[ii] = ii/sqrt(N);
}
// tamaños de vector
for( int k=0 ; k<NUM_TAM; k++) { // VECTORES DE DISTINTOS TAMAÑOS
    N=tam[k];
    printf(" tamaño de vector N = %d \n", N);
    for (int iter=0; iter< MAX_ITER; iter++){ //ITERACIONES
        //producto SIMD
        gettimeofday(&ini,NULL);
        answer[iter] = dot_product(a, b, N);
        gettimeofday(&fin,NULL);
        tiempo[iter]=(fin.tv_sec*1000000+fin.tv_usec)-(ini.tv_sec*1000000+ini.tv_usec);
        printf("iter = %d ,resultado SIMD =%f y tiempo SIMD: %ld us. \n", iter, answer[iter], tiempo[iter]);

        // producto secuencial escalar
        gettimeofday(&ini,NULL);
        answerdot[iter] = slow_dot_product(a, b, N);
        gettimeofday(&fin,NULL);
        tiempodot[iter]=(fin.tv_sec*1000000+fin.tv_usec)-(ini.tv_sec*1000000+ini.tv_usec);
        rev_answerdot[iter] = reverse_slow_dot_product(a, b, N);
        acel=(float)tiempodot[iter]/(float)tiempo[iter];
        printf("iter = %d ,result sec1 =%f, result sec2 =%f y tiempo sec: %ld us. acel =%f \n", iter, answer[iter], answerdot[iter], tiempo[iter], tiempodot[iter], rev_answerdot[iter], acel);
    }
}
}

```

La compilación del código SIMD se debe realizar con CFLAGS := -O3 -march=native

Double-click (or enter) to edit

```
!gcc exdot_product.c -O3 -lm -march=native -twofile-program -Wall -D_GNU_SOURCE -o
```

```
! ./exdot_producto
```

[Show hidden output](#)

✓ Ejercicios

Compruebe los resultados del producto escalar

- E2.1 ¿Coinciden siempre los resultados SIMD con el secuencial?
 - E2.2 ¿Coincide los denominados secuencial 1 a secuencial 2?
 - E2.3 ¿Cómo se justifican estos resultados?
 - E2.4 Si son diferentes ¿Cuál es el resultado correcto?
-
- E2.1: Los resultados SIMD coinciden de forma aproximada con el secuencial, la desviación de precisión decimal es poco notable, con lo cual se podría estimar que los resultados obtenidos son equivalentes.
 - E2.2: Los resultados de sec1 y sec2 son como en el caso anterior casi iguales pero no idénticos, ya que cuanto más incrementa el número de iteraciones los resultados son diferentes en las milésimas.
 - E2.3: Los resultados muestran diferencias debido a las distintas formas que se emplean en SIMD y en las operaciones escalares para redondear decimales y no perder precisión, SIMD utiliza aritmética de precisión mientras que las operaciones tradicionales es escalares usan aritmética modular. Además dentro de las secuenciales hay también diferencia puesto que una de las funciones escalares (reverse_slow_dot_product) empieza operando por los números más grandes.
 - E2.4: La mayor precisión la tiene en este caso slow_dot_product puesto que es la escalar que utiliza aritmética modular y comienza a operar por los números decimales más pequeños con lo cual no se pierde precisión al redondear al inicio, al contrario que en la otra función escalar, por otro lado, las operaciones vectoriales aunque más eficientes en tiempo, conllevan una pérdida decimal por ende la función escalar slow_dot_product sería la más precisa.

Tiempo de ejecución según opciones de optimización

La compilación del código SIMD por defecto se realiza con CFLAGS := -O3 -march=native

✓ Estudio mas detallado del producto escalar de dos vectores con elementos reales (double) optimizado con SIMD

En este ejercicio, a modo de tutorial, se desarrolla el Producto escalar SIMD de dos vectores con elementos reales (double).

- Se compara el tiempo de ejecucion SIMD con el equivalente secuencial no paralelizado
- Se comparan resutados y tiempos de ejecución con distintas opciones de compilación.

```
%%writefile dot_product.c

#ifndef ITER
#define ITER 25
#endif

#ifndef ARRAYSIZE
#pragma message("ARRAYSIZE not defined. Check compile command")
#define ARRAYSIZE 1000000
#endif

#ifndef REPS
#define REPS 1
#endif

// #define STATIC_MEM

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <sys/time.h>
#include <immintrin.h>

typedef unsigned long long ull;

void initarray(double *a, ssize_t n) {
    for (ssize_t i=0; i<n; i++) {
        a[i] = rand() / (rand() + 1);
    }
}

static inline double naive_dot_product(double* a, double* b, ssize_t n) {
    double res = 0.0;
    ssize_t i = 0;
    for (i=0; i<n; i++) {
        res += a[i]*b[i];
    }
}
```

```

    return res;
}

static inline double reversed_dot_product(double* a, double* b, ssize_t n) {
    double res = 0.0;
    ssize_t i = 0;
    for (i=n-1; i>=0; i--) {
        res += a[i]*b[i];
    }
    return res;
}

/* Horizontal add works within 128-bit lanes. Use scalar ops to add
 * across the boundary. */
static inline double reduce_vector1(__m256d input) {
    __m256d temp = (__m256d) _mm256_hadd_pd(input, input);
    return ((double*)&temp)[0] + ((double*)&temp)[2];
}

/* Another way to get around the 128-bit boundary: grab the first 128
 * bits, grab the lower 128 bits and then add them together with a 128
 * bit add instruction. */
static inline double reduce_vector2(__m256d input) {
    __m256d temp = _mm256_hadd_pd(input, input);
    __m128d sum_high = _mm256_extractf128_pd(temp, 1);
    __m128d result = _mm_add_pd(sum_high, _mm256_castpd256_pd128(temp));
    return ((double*)&result)[0];
}

static inline double avx_dot_product(const double *a, const double *b, ssize_t n)
    __m256d sum_vec = _mm256_set_pd(0.0, 0.0, 0.0, 0.0);

    /* Add up partial dot-products in blocks of 256 bits */
    for(ssize_t ii = 0; ii < n ; ii += 4) {
        __m256d x = _mm256_load_pd(a+ii);
        __m256d y = _mm256_load_pd(b+ii);
        __m256d z = _mm256_mul_pd(x,y);
        sum_vec = _mm256_add_pd(sum_vec, z);
    }

    /* Find the partial dot-product for the remaining elements after
     * dealing with all 256-bit blocks. */
    double final = 0.0;
    for(ssize_t ii = n - (n%4); ii < n; ++ii)
        final += a[ii] * b[ii];

    return reduce_vector2(sum_vec) + final;
}

```

```
int main(int argc, char **argv) {
```

```

int main(int argc, char **argv) {

    if(argc < 2 || argc > 2){
        printf(" uso: dot_product dotproductdata.txt \n");
        return 0;
    }
    const char *filename = argv[1];
    FILE *fp = fopen(filename,"a");
    if (fp == NULL) {
        perror(filename);
        return -1;
    }

#ifdef STATIC_MEM
    __attribute__((aligned (32))) double a[ARRAYSIZE], b[ARRAYSIZE];
#else
    double *a, *b;
    a = aligned_alloc(256, ARRAYSIZE*sizeof(double));
    b = aligned_alloc(256, ARRAYSIZE*sizeof(double));
#endif

    struct timespec ini, fin;
    ull time_naive = 0, time_avx=0;
    initarray(a, ARRAYSIZE);
    initarray(b, ARRAYSIZE);
    double answers_naive[REPS]={0}, answers_avx[REPS]={0};
    double answer_naive=0, answer_avx=0;
    double speedup = 0;
    for (ull iter=0; iter < ITER; iter++) {

        #define CLOCK_METHOD CLOCK_PROCESS_CPUTIME_ID

        clock_gettime( CLOCK_METHOD, &ini); // gettimeofday(&ini,NULL);
        for (ull r=0; r < REPS; r++)
            answers_naive[r] = naive_dot_product(a, b, ARRAYSIZE);
        clock_gettime( CLOCK_METHOD, &fin); // gettimeofday(&fin,NULL);
        for (ull r=0; r < REPS; r++)
            answer_naive += answers_naive[r];
        answer_naive = answer_naive / REPS;
        time_naive = (fin.tv_sec*1000000000+fin.tv_nsec)-(ini.tv_sec*1000000000+ini.tv_nsec);

        clock_gettime( CLOCK_METHOD, &ini); // gettimeofday(&ini,NULL);
        for (ull r=0; r < REPS; r++)
            answers_avx[r] = avx_dot_product(a, b, ARRAYSIZE);
        clock_gettime( CLOCK_METHOD, &fin); // gettimeofday(&fin,NULL);
        for (ull r=0; r < REPS; r++)
            answer_avx += answers_avx[r];
        answer_avx = answer_avx / REPS;
        time_avx = (fin.tv_sec*1000000000+fin.tv_nsec)-(ini.tv_sec*1000000000+ini.tv_nsec);

        speedup = ((double) time_naive) / ((double)time_avx);
    }
}

```

```

    fprintf(fp,"%llu %u %lf %llu %lf %llu %lf\n", iter, ARRAYSIZE, answer_naive, 1
}

#ifdef STATIC_MEM
    free(a);
    free(b);
#endif
fclose(fp);
}

```

Show hidden output

```
!gcc dot_product.c -O3 -g -fopt-info -DREPS=10 -DARRAYSIZE={i} -lm -march=native
```

Show hidden output

```

!echo "iteracion tamaño res_naive time_naive res_simd time_simd speedup" > dotproc
!echo "iteracion tamaño res_naive time_naive res_simd time_simd speedup" > dotproc
!echo "iteracion tamaño res_naive time_naive res_simd time_simd speedup" > dotproc

for i in tam:
    !gcc dot_product.c -g -DREPS=10 -DARRAYSIZE={i} -lm -march=native -fwhole-program
    !gcc dot_product.c -g -std=gnu11 -DREPS=10 -O3 -DARRAYSIZE={i} -lm -march=native
    !gcc dot_product.c -g -DREPS=10 -O1 -DARRAYSIZE={i} -lm -march=native -fwhole-program
    !./dot_producto dotproductdata00.txt
    !./dot_producto03 dotproductdata03.txt
    !./dot_producto01 dotproductdata01.txt

! cat dotproductdata00.txt | head -n 30

```

Show hidden output

```

import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

df0 = pd.read_csv("dotproductdata00.txt", sep=" ")
for c in df0.columns:
    df0[c] = pd.to_numeric(df0[c])

sns.boxplot(df0, x="tamaño", y="speedup")
plt.xlabel("Tamaño (palabras)")
plt.xticks(rotation=45)
plt.ylabel("Speedup");
plt.ylim([0,5])

```

Show hidden output

En los diagrama de cajas podemos ver como los resultados del speedup a medida que aumentamos el tamaño no varía notablemente. Existen algunos valores poco comunes, pero en la gran mayoría el speedup no aumenta. Esto podría indicar que el tamaño no influye a la hora de calcular el speedup.

```
sns.scatterplot(df0, x="tamaño", y="speedup")
plt.xlabel("Tamaño (palabras)")
plt.xticks(rotation=45)
plt.ylabel("Speedup");
```

Show hidden output

```
df3 = pd.read_csv("dotproductdata03.txt", sep=" ")
```

```
sns.boxplot(df3, x="tamaño", y="speedup")
plt.xlabel("Tamaño (en palabras de 8B)")
plt.xticks(rotation=45)
plt.ylabel("Speedup");
plt.ylim([0,5])
```

Show hidden output

En la gráfica observamos que a medida que se aumenta el tamaño el speedup va siendo más lento. Podemos ver que cuando los valores son más pequeños el speedup toma distintos valores, pero a medida que va creciendo el tamaño empieza a tomar valores más constantes.

Comparación con diferentes niveles de optimización del compilador Habilitar los diferentes niveles de optimización del compilador (p. ej., -O1, -O2 y -O3). Compare los tiempos de ejecución de las diferentes versiones optimizadas según el nivel de optimización. ¿El código optimizado es más rápido? Si es así, ¿cuánto?

```
df1 = pd.read_csv("dotproductdata03.txt", sep=" ")
```

```
df0["Opt"] = "O0"
df3["Opt"] = "O3"
df1["Opt"] = "O1"
```

```
df = pd.concat([df0, df3, df1])
```

```
sns.barplot(df, x="tamaño", y="time_naive", hue="Opt")
```

```
plt.title('Tiempo (sin SIMD) vs tamaño del vector (optimización -O3 vs -O0 vs -O1)')
plt.xlabel('Tamaño (palabras)')
plt.ylabel('Tiempo (us)')
plt.xticks(rotation=45)
plt.yscale('log')
```

Show hidden output

```
sns.barplot(df, x="tamaño", y="time_simd", hue="Opt")
plt.title('Tiempo AVX vs tamaño del vector (optimización -O3 vs -O0 vs -O1)')
plt.xlabel('Tamaño (palabras)')
plt.ylabel('Tiempo (us)')
plt.xticks(rotation=45)
plt.yscale('log');
```

Show hidden output

```
sns.barplot(df, x="tamaño", y="speedup", hue="Opt")
plt.title('Speedup vs tamaño del vector (optimización -O3 vs -O0 vs -O1)')
plt.xlabel('Tamaño (palabras)')
plt.ylabel('Speedup')
plt.xticks(rotation=45)
plt.ylim([0,6]);
```

Show hidden output

Para los tamaños más grandes de los vectores podemos observar que cuando el nivel de optimización es mayor el tiempo de aceleración es menor. Sin embargo cuando la optimización es O1 y O3 son resultados similares. Esto se debe a que cuando se ejecuta el programa sin optimización, cuando se vectoriza el vector el margen de mejora es muy amplio por lo que el speedup sale más grande. Por otro lado al realizar ya una optimización como O3, cuando se vectoriza el margen de mejora es menor por que el programa ya está optimizado por eso el speedup es menor.

En las gráficas donde se comparan los tiempos de AVX y los tiempos sin SIMD podemos ver que para los tiempos de AVX el tiempo es menor. Esto puede deberse a que AVX puede procesar en paralelo muchos datos.

En el último barplot vemos que más o menos cuando el vector no está optimizado tarda lo mismo sin importar el tamaño, y cuando tiene algún grado de optimización este tiempo se reduce.

Ejercicio 3: Producto escalar de dos vectores con elementos

✓

float de precisión simple (32 bit)

floats de precisión simple (32 bit).

Basándose en el producto escalar de dos vectores con elementos reales (double de 64 bits), desarrolle y pruebe de manera similar el producto escalar de dos vectores con elementos cuyos valores son float de 32 bit (valores reales de coma flotante en precisión simple).

Al igual que en el tutorial debe realizar con criterio un estudio de la aceleración obtenida al vectorizar para diferentes tamaños de vector y comparar con resultados en un equipo de laboratorio o en su equipo personal, comentando los resultados obtenidos y destacando las principales diferencias observadas.

```
%%writefile float_product.c

#ifndef ITER
#define ITER 25
#endif

#ifndef ARRAYSIZE
#pragma message("ARRAYSIZE not defined. Check compile command")
#define ARRAYSIZE 1000000
#endif

#ifndef REPS
#define REPS 1
#endif

// #define STATIC_MEM

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <sys/time.h>
#include <immintrin.h>

typedef unsigned long long ull;

void initarray(float *a, ssize_t n) {
    for (ssize_t i = 0; i < n; i++) {
        a[i] = (float)(rand() / (rand() + 1.0));
    }
}

static inline float naive_dot_product(float* a, float* b, ssize_t n) {
    float res = 0.0f;
    for (ssize_t i = 0; i < n; i++) {
        res += a[i] * b[i];
    }
}
```

```

    return res;
}

static inline float reversed_dot_product(float* a, float* b, ssize_t n) {
    float res = 0.0f;
    for (ssize_t i = n - 1; i >= 0; i--) {
        res += a[i] * b[i];
    }
    return res;
}

/* Horizontal add works within 128-bit lanes. Use scalar ops to add
 * across the boundary. */
static inline float reduce_vector1(__m256 input) {
    __m256 temp = _mm256_hadd_ps(input, input);
    return ((float*)&temp)[0] + ((float*)&temp)[4];
}

/* Another way to get around the 128-bit boundary: grab the first 128
 * bits, grab the lower 128 bits and then add them together with a 128
 * bit add instruction. */
static inline float reduce_vector2(__m256 input) {
    __m256 temp = _mm256_hadd_ps(input, input);
    __m128 sum_high = _mm256_extractf128_ps(temp, 1);
    __m128 result = _mm_add_ps(sum_high, _mm256_castps256_ps128(temp));
    return ((float*)&result)[0];
}

static inline float avx_dot_product(const float *a, const float *b, ssize_t n) {
    __m256 sum_vec = _mm256_set_ps(0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f);

    /* Add up partial dot-products in blocks of 256 bits */
    for (ssize_t ii = 0; ii < n; ii += 8) {
        __m256 x = _mm256_load_ps(a + ii);
        __m256 y = _mm256_load_ps(b + ii);
        __m256 z = _mm256_mul_ps(x, y);
        sum_vec = _mm256_add_ps(sum_vec, z);
    }

    /* Find the partial dot-product for the remaining elements after
     * dealing with all 256-bit blocks. */
    float final = 0.0f;
    for (ssize_t ii = n - (n % 8); ii < n; ++ii)
        final += a[ii] * b[ii];

    return reduce_vector2(sum_vec) + final;
}

int main(int argc, char **argv) {
    if (argc < 2 || argc > 2) {

```

```

11 (argc < 2 || argc > 2) {
    printf(" uso: dot_product dotproductdata.txt \n");
    return 0;
}
const char *filename = argv[1];
FILE *fp = fopen(filename, "a");
if (fp == NULL) {
    perror(filename);
    return -1;
}

#ifdef STATIC_MEM
    __attribute__((aligned (32))) float a[ARRAYSIZE], b[ARRAYSIZE];
#else
    float *a, *b;
    a = aligned_alloc(256, ARRAYSIZE * sizeof(float));
    b = aligned_alloc(256, ARRAYSIZE * sizeof(float));
#endif

struct timespec ini, fin;
ull time_naive = 0, time_avx = 0;
initarray(a, ARRAYSIZE);
initarray(b, ARRAYSIZE);
float answers_naive[REPS] = {0}, answers_avx[REPS] = {0};
float answer_naive = 0, answer_avx = 0;
double speedup = 0;
for (ull iter = 0; iter < ITER; iter++) {

    #define CLOCK_METHOD CLOCK_PROCESS_CPUTIME_ID

    clock_gettime(CLOCK_METHOD, &ini);
    for (ull r = 0; r < REPS; r++)
        answers_naive[r] = naive_dot_product(a, b, ARRAYSIZE);
    clock_gettime(CLOCK_METHOD, &fin);
    for (ull r = 0; r < REPS; r++)
        answer_naive += answers_naive[r];
    answer_naive = answer_naive / REPS;
    time_naive = (fin.tv_sec * 1000000000 + fin.tv_nsec) - (ini.tv_sec * 1000000000

    clock_gettime(CLOCK_METHOD, &ini);
    for (ull r = 0; r < REPS; r++)
        answers_avx[r] = avx_dot_product(a, b, ARRAYSIZE);
    clock_gettime(CLOCK_METHOD, &fin);
    for (ull r = 0; r < REPS; r++)
        answer_avx += answers_avx[r];
    answer_avx = answer_avx / REPS;
    time_avx = (fin.tv_sec * 1000000000 + fin.tv_nsec) - (ini.tv_sec * 1000000000

    speedup = ((double)time_naive) / ((double)time_avx);
    fprintf(fp, "%llu %u %f %llu %f %llu %lf\n", iter, ARRAYSIZE, answer_naive, t
}

```

```

#ifdef STATIC_MEM
    free(a);
    free(b);
#endif
fclose(fp);
}

```

Writing float_product.c

```

!echo "iteracion tamaño res_naive time_naive res_simd time_simd speedup" > floatp
!echo "iteracion tamaño res_naive time_naive res_simd time_simd speedup" > floatp
for i in tam[:3]:
    !gcc float_product.c -g -DREPS=10 -DARRAYSIZE={i} -lm -march=native -fwhole-prog
    !gcc float_product.c -g -std=gnu11 -DREPS=10 -O3 -DARRAYSIZE={i} -lm -march=nati
    !./float_product floatproductdata00.txt
    !./float_product03 floatproductdata03.txt

! cat floatproductdata00.txt | head -n 30
! cat floatproductdata03.txt | head -n 30

```

[Show hidden output](#)

```

import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

df0 = pd.read_csv("floatproductdata00.txt", sep=" ")
for c in df0.columns:
    df0[c] = pd.to_numeric(df0[c])

sns.boxplot(df0, x="tamaño", y="speedup")
plt.xlabel("Tamaño (palabras)")
plt.xticks(rotation=45)
plt.ylabel("Speedup");
plt.ylim([0,5])

```

[Show hidden output](#)

```

sns.scatterplot(df0, x="tamaño", y="speedup")
plt.xlabel("Tamaño (palabras)")
plt.xticks(rotation=45)
plt.ylabel("Speedup")

```

[Show hidden output](#)

```

df3 = pd.read_csv("floatproductdata03.txt", sep=" ")

```

```
sns.boxplot(df3, x="tamaño", y="speedup")
plt.xlabel("Tamaño (en palabras de 8B)")
plt.xticks(rotation=45)
plt.ylabel("Speedup");
plt.ylim([0,5])
```

[Show hidden output](#)

En la gráfica podemos observar que cuando el tamaño del vector es pequeño tienen una aceleración mayor, mientras que cuanto más grande es el tamaño del vector menor es la aceleración. Esto podría deberse a que cuando el vector es más grande la vectorización es más ineficiente dado el desbordamiento de las cachés de la CPU lo que aumenta el número de accesos tanto a las cachés como a la memoria principal. Podemos ver al igual que cuando se calculaba con el código anterior, que a medida que el tamaño aumenta el speedup se vuelve más constante.

```
df0["Opt"] = "00"
df3["Opt"] = "03"
```

```
df = pd.concat([df0, df3])
```

```
sns.barplot(df, x="tamaño", y="time_naive", hue="Opt")
plt.title('Tiempo (sin SIMD) vs tamaño del vector (optimización -03 vs -00)')
plt.xlabel('Tamaño (palabras)')
plt.ylabel('Tiempo (us)')
plt.xticks(rotation=45)
plt.yscale('log');
```

[Show hidden output](#)

```
sns.barplot(df, x="tamaño", y="time_simd", hue="Opt")
plt.title('Tiempo AVX vs tamaño del vector (optimización -03 vs -00)')
plt.xlabel('Tamaño (palabras)')
plt.ylabel('Tiempo (us)')
plt.xticks(rotation=45)
plt.yscale('log');
```

[Show hidden output](#)

Double-click (or enter) to edit

```
sns.barplot(df, x="tamaño", y="speedup", hue="Opt")
plt.title('Speedup vs tamaño del vector (optimización -03 vs -00)')
```

```
plt.title('Speedup vs tamaño del vector (optimización -03 vs -00)')  
plt.xlabel('Tamaño (palabras)')  
plt.ylabel('Speedup')  
plt.xticks(rotation=45)  
plt.ylim([0,6]);
```

[Show hidden output](#)

Como con las gráficas cuando hacíamos el producto vectorial con enteros tipo double, los resultados cuando no se usa optimización son bastante distintos a los resultados cuando se usa optimización. Esto se debe a que cuando no usamos la optimización el margen de mejora cuando se vectoriza es muy amplio, por lo que el speedup sale muy alto, mientras que cuando optimizamos ya hemos mejorado el rendimiento, por lo que cuando vectorizamos, el margen de mejora es menor, por eso salen resultados más pequeños.